

OBJECT ORIENTED MODELING AND DESIGN(CXT 309)

Credit-3

MODULE 1 Object modeling

Text:1. Object Oriented Modeling and Design - James Rumbaugh, 1 st edition, Prentice Hall India

Module - 1 (Object modeling)

- Introduction: Object Oriented Development - Modeling Concepts–Object Oriented Methodology. Object Oriented Themes - Abstraction - Encapsulation - Combining Data and Behavior - Sharing-Emphasis on Object Structure.
- Object modeling: Objects and Classes, Links and Associations, Advanced links and Association Concepts, Generalization and Inheritance, Grouping Constructs, A Sample Object Model.

2

Software development life cycle

- **Software development life cycle (SDLC) is a structured process that is used to design, develop, and test good-quality software.**
- SDLC, or software development life cycle, is a methodology that defines the entire procedure of software development step-by-step.

3

- Intention of object oriented modeling and design is to learn how to apply object -oriented concepts to all the stages of the software development life cycle.
- Object oriented modeling and design is a new way of thinking about problems using models organized around real world concepts.
- The fundamental construct is the object, which combines both data structure and behavior.

4

Object

Data can be quantized into discrete, distinguishable entities which are called objects.

Object Oriented Approach means organizing software as a collection of discrete objects that incorporate both data structure and behavior.

Model

An abstraction of something for the purpose of understanding it before building it

Designing

Grady Booch-OOSD

Ivar Johnson-OOSE

James Raumbaugh-OMT

UML-Unified Modeling Language

5

Object Oriented Development is a new way of thinking about software based on abstractions that exist in the real world as well as in the program.

Object Oriented Methodology is a methodology for object oriented development and a graphical notation for representing objects oriented concepts.

We can call this methodology as **OMT**(Object Modeling Technique)

The methodology has the following **stages**:

- **Analysis**
- **System design**
- **Object design**
- **Implementation**

7

Analysis

- An analysis model is a concise, precise abstraction of **what the desired system must do, not how it will be done.**
- It **should not contain any implementation decisions.**
- The **objects** in the model should be **application domain concepts** and **not the computer implementation concepts**(datastructures).

8

System design

- The designer makes **high level decisions** about the overall architecture.
- In system design, the **target system** is organized as various **subsystems** based on both the **analysis structure** and the **proposed architecture**.

9

Object design

- The designer builds a **design model based on the analysis model but containing implementation details**.
- The **focus** of object design is the **data structures and algorithms** needed to implement each cycle.

10

Implementation

- The **object classes and relationships** developed during object design are finally **translated** into a particular **programming language, database, or hardware implementation**.
- During implementation, it is important to **follow good software engineering practice** so that the system can remain the traceability, flexibility and extensibility.

11

- The **OMT methodology** uses **three** kinds of **models** to describe a system.

- Object Model
- Dynamic Model
- Functional Model

12

Object Model

- Describes the **static structure** of the **objects** in a system and their **relationships**.
- Main concepts are **classes** and **associations**, with attributes and operations.
- **Aggregation** and **generalization** are predefined relationships
- This model mainly contains **object diagrams**.
- An object diagram is a graph whose nodes are object classes and whose arcs are relationships among classes.

13

Dynamic Model

- **The dynamic model** describes the **aspects of a system that change over time**.
- It is used to **specify and implement the control aspects** of a system
- It represents a state/transition view on the model.
- Main concepts are states, transitions between states and events to trigger transitions.
- Actions can be modeled as occurring within states.
- This model contains **state diagrams**.
- A state diagram is a graph whose **nodes are states** and whose **arcs are transitions between states** caused by events.

14

Functional Model

- describes the **data value transformations within a system**.
- Main concepts are process, data store, data flow, and actors.
- This model contains the **data flow diagrams**
- A data flow diagram represents a computation.
- A data flow diagram is a graph whose nodes are processes and whose arcs are dataflows.

15

(v) Object Oriented Themes:

- There are several themes in an object oriented technology.
 - These themes are not unique to object oriented systems.
1. **Abstraction**
 2. **Encapsulation**
 3. **Combining Data and Behavior**
 4. **Sharing**
 5. **Emphasis on the essence of object**

16

1. Abstraction

- Abstraction consists of **focusing** on the **essential, inherent aspects** of an entity and **ignoring** its **accidental properties**.
- In system development, it focuses only **on what an object is and does**, before deciding how it should be implemented.
- Use of abstraction during analysis means **dealing only with application domain concepts**, not making decisions before problem is understood.
- Proper use of abstraction allows the **same model** to be used for **analysis, high level design, program structure, database structure and documentation**.

17

2. Encapsulation

- It can also be called as **information hiding**.
- It consists of **separating the external aspects of an object**, which are accessible to other objects, **from the internal implementation details of the object**, which are hidden from other objects.
- It **prevents** program from **becoming** so **interdependent** that a small change has massive ripple effect.
- The **implementation** of an object can be **changed without affecting the applications** that use it.
- It gives the ability to **combine data structure and behavior** in a single entity.

18

3. Combining Data and Behavior

- The **caller** of an operation **need not consider how many implementations** of a given operation exist.
- **Operator polymorphism** shifts the burden of deciding what implementation to use from the calling code to the class hierarchy.
- Eg: Consider an object oriented program calling a draw procedure for drawing different figures say a polygon, circle, or text. The decision of which procedure to use is made by each object, based on its class.
- In an object oriented system, the **data structure hierarchy** is **identical** to the **operation inheritance hierarchy**.

19

4. Sharing

- Inheritance of both data structure and behavior allows a **common structure** to be **shared** among several similar subclasses **without redundancy**.
- The **sharing of code using inheritance** is one of the main advantages of object oriented languages.
- OO development offers the prospect of **reusing designs and code on future projects**.

20

5. Emphasis on the essence of object

- Object Oriented Technology stresses **what an object** is, rather than how it is used.
- The **uses** of an object **depend** on the **details** of the **application** and often **change during development**.
- As requirements evolve, the features supplied by an object are much more stable than the ways it is used, hence **software systems built on object structure are more stable in the long run**.
- **OO development** places a **greater emphasis** on **data structure** and a lesser emphasis on procedure structure

21

(vi) Object Modeling

- A model is an **abstraction** of something for the purpose of **understanding** it **before building** it.
- It is **easier to manipulate than the original entity**.
- An object model captures the **static** structure of a system, **relationships** between the objects, the **attributes and operations** that characterize each class of objects.
- This model is the most important of the three models.

22

Some basic concepts in object modeling:

(a) Objects

- Purpose of object modelling is to describe objects.
- An object is something that **make sense in an application context**.
- We can define an object as a **concept, abstraction, or thing with crisp boundaries and meaning for the problem at hand**.
- Eg: Smith, Simplex company, process number 348
- Objects serve two purposes:
 - They promote **understanding of the real world**
 - Provide a **practical basis for computer implementation**
- All objects have **identity** and are **distinguishable**

23

(b) Classes

- An 'object class' or 'class' describes a **group of objects with similar properties(attributes), common behavior(operations), common relationships to other objects, and common semantics**.
- Eg: Person, company, animal, process
- Each **object knows its class**.
- An object's class is an implicit property of the object

24

(c) Object Diagrams

- They provide a formal graphic notation for modeling objects, classes and their relationships to one another.
- They are useful both for abstract modeling and for designing actual program.
- There are two types of object diagrams
 - Class diagrams
 - Instance diagrams

25

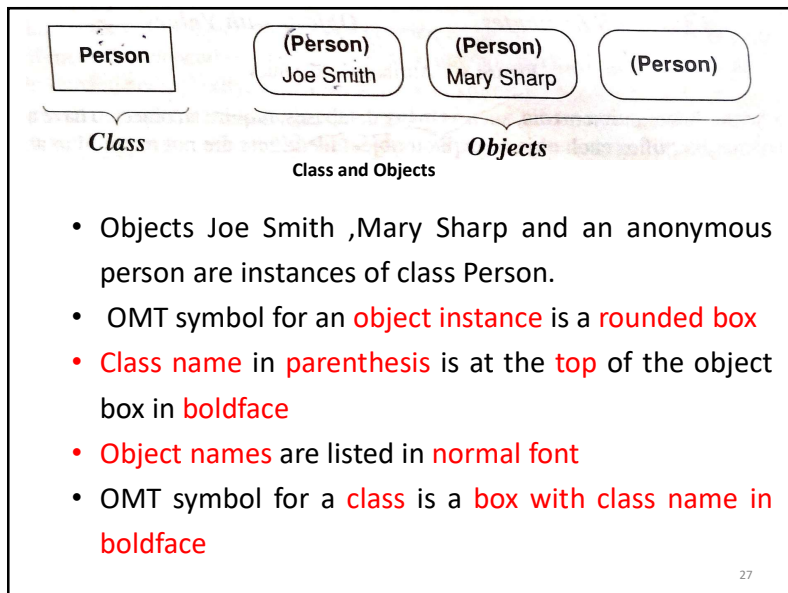
Class Diagram:

- It is a schema, pattern or template for describing many possible instances of data.
- A class diagram describes object classes
- A given class diagram corresponds to an infinite set of instance diagrams

Instance Diagram:

- It describes how a particular set of objects relate to each other.
- An instance diagram describes object instances.
- Instance diagrams are useful for documenting test cases(scenarios) and discussing examples.

26



27

(d) Attributes

- An attribute is a data value held by objects in a class.
- Eg: Name, age, etc. of a person are attributes
- Each attribute is unique within a class
- Attributes are listed in the second part of the class box.
- Each attribute name may be followed by optional details, such as type and default value.
- Class boxes have a line drawn between the class name and attributes

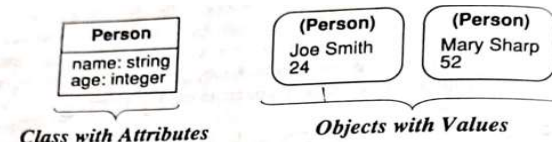


Figure 3.2 Attributes and values

28

(e) **Operations and methods:**

- An operation is a **function or transformation** that may be **applied to or by objects** in a class.
- Eg: Hire, fire, pay-dividend are operations on class company
- **All objects** in a **class** share the **same operations**
- Each **operation** has a **target object** as an **implicit argument**
- A **method** is the **implementation** of an **operation** for a **class**.
- eg:
 - Class *File* may have an operation *print*.
 - Different methods could be implemented to print ASCII file, print binary files, print digitized picture files
- An **operation** may have **arguments** in addition to its target object.

29

- Operations are listed in the lower third part of the class box.

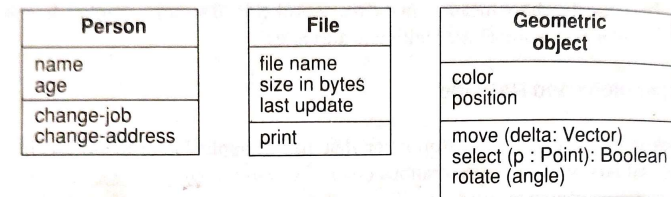


Figure 3.4 Operations

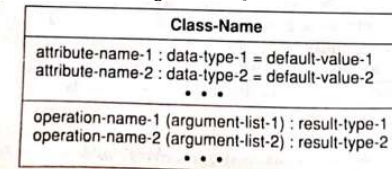


Figure 3.5 Summary of object modeling notation for classes

30

Links and association concepts

- They are used for **establishing relationships** among **objects and classes**.

1) Links and Associations**Link**

- A link is a **physical or conceptual connection** between **object instances**.
- A link is an **instance of an association**.
- A link is drawn as a line between objects
- Eg: - Joe Smith works for Simplex company.

31

Association

- Describes a **group of links with common structure and common semantics**
- Eg: A person *works for* a company.
- All the links in an association connect objects from the same class.
- Associations are inherently **bidirectional**.
- It means that a binary association can be traversed in both directions.
- They are implemented as **pointers from one object to another**
- OMT notation is a line between classes

32

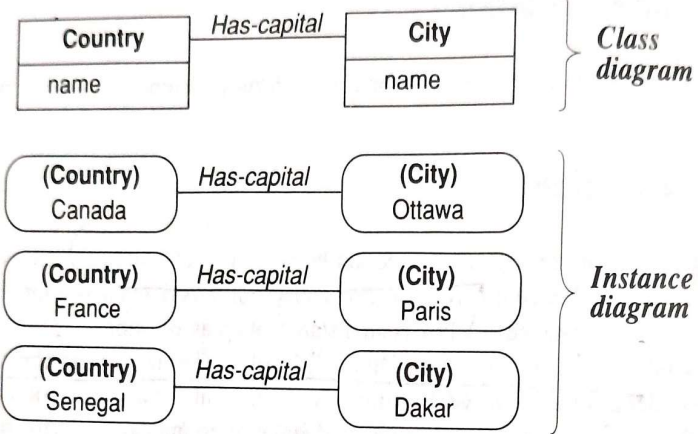
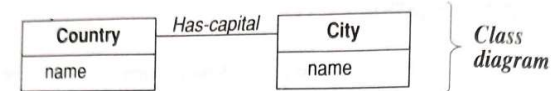


Figure 3.6 One-to-one association and links

33

One to one association and links



34



Instance diagram

35

many to many association and links

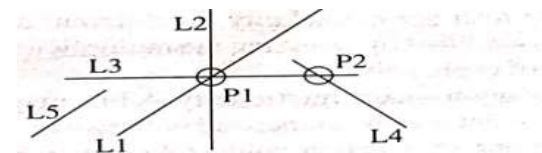
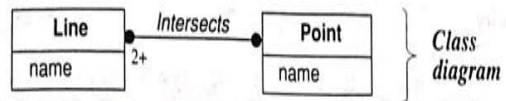


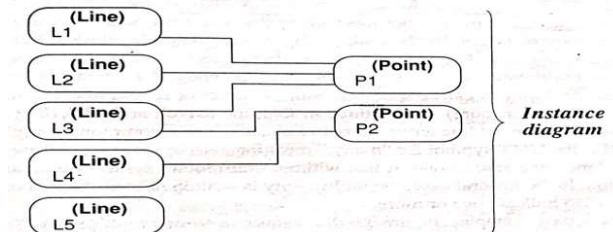
Figure 3.7 Many-to-many

- A line may have zero or more intersection points.
- An intersection point may be associated with two or more lines.
- In this particular case, L1, L2, and L4 have one intersection point.
- L3 has two intersection points.
- L5 has no intersection points. P1 intersects with three lines. P2 intersects with two lines.

36



37



38

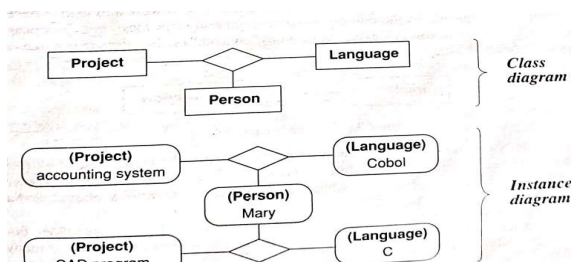
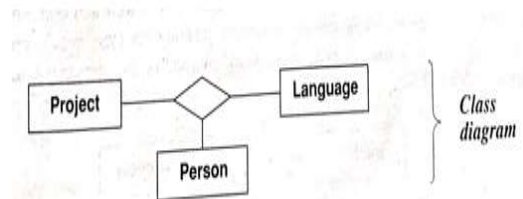


Figure 3.8 Ternary association and links

39

Ternary association and links



- Persons who are programmers use computer languages on projects.

40

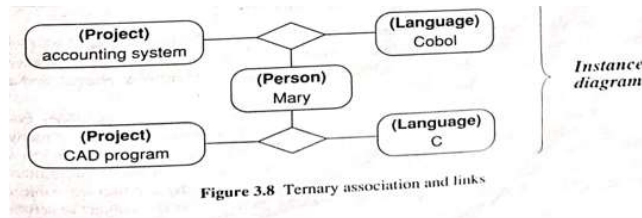


Figure 3.8 Ternary association and links

- This ternary association is atomic and cannot be subdivided into binary associations without losing information.
- The symbol for general ternary and n-ary associations is a diamond with lines connecting to related classes

41

2) Multiplicity

- Multiplicity specifies **how many instances of one class may relate to each instance of another class**.
- Multiplicity constrains number of related objects
- The solid balls(0 or more),hollow ball(0 or 1),”2+” are multiplicity symbols.(1 exactly one ,1+ one or more,3-5 three to five inclusive, 2,4,18 means 2 , 4 or 18)

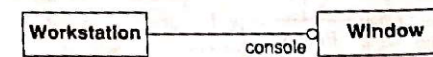


Figure 3.9 Zero-or-one multiplicity

- Associations may be binary, ternary or higher order.
- A work station may have one of its windows designated as the console to receive general error messages. It is possible, that no console window exists.(The word console on the diagram is a role name.)

42

(g)Advanced Link and Association Concepts:

i) Link Attributes

- A link attribute is a **property** of the **links** in an association.
- Eg: access permission is an attribute of 'Accessible by'.

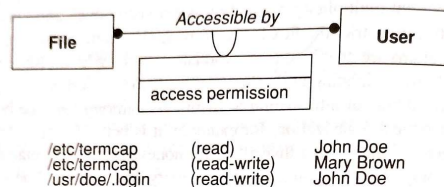


Figure 3.10 Link attribute for a many-to-many association

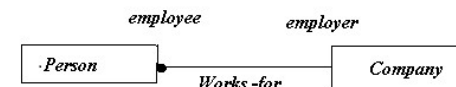
- The **OMT notation** for a link attribute is a **box attached to the association by a loop**

43

ii) Role names

- A role is an **end** of an **association**.
- Role name is a name that **uniquely identifies one end of an association**.
- Eg: - “Person works for a Company.”

Here the person plays the role of an **employee** and the company the role of the **employer**



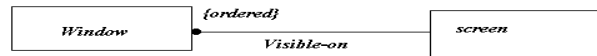
Role names for an association

44

iii) Ordering

- Sometimes, the objects are explicitly ordered.
- The ordering is an **inherent** part of the **association**
- An ordered set of objects on the “many” end of an association is indicated by writing “**{ordered}**” next to the multiplicity dot for the role.

Eg: A workstation screen containing a number of overlapping windows. The windows are explicitly ordered, so only the topmost window is visible at any point on the screen.



Ordered sets in an association

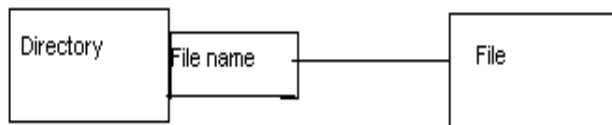
45

iv) Qualification

- A qualifier is an **association attribute**.
- A qualified association **relates two object classes** and a **qualifier**
- Qualifier is a special **attribute** that **reduces the effective multiplicity of an association**.
- One-to-many and many-to-many associations may be qualified
- Qualification **improves semantic accuracy** and **increases the visibility of navigation paths**.
- The qualifier distinguishes among the **set of objects at the many end** of an association.

46

- Eg: - A directory has many files. A file may only belong to a single directory. Within the context of a directory, the file name specifies a unique file. A file corresponds to a directory and a file name.



A qualified association

47

- It is much more informative to be told that a directory and file name combine to identify a file, rather than be told that a directory has many files.
- The qualification syntax also indicates that each file name is unique within directory.
- A qualifier is drawn **as small box on the end of the association line near the class it qualifies**.
- The qualifier box is part of the association, not part of the class.
- The qualifier attributes are drawn within the qualifier box.
- Directory + Filename yields a File, therefore file name is listed in a box contiguous to Directory.

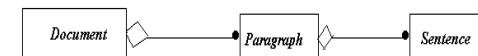
48

- When you add a qualifier to an association end, it narrows down its multiplicity.
- Eg: the relationship between classes Bank and Cashier has a multiplicity of one-to-many, i.e., one Bank employs many Cashiers.
- The addition of a qualifier called Employee Code would reduce the multiplicity to one-to-one, i.e., a Bank plus an Employee Code corresponds to a particular Cashier.

49

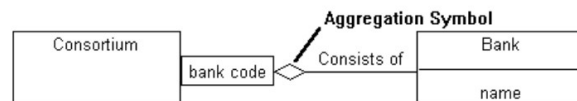
(v) Aggregation

- Aggregation is the “part-whole” or “a-part-of” relationship in which objects representing the components of something are associated with an object representing the entire assembly.
- The class opposite to the diamond side is part of the class on the diamond side.
- Eg: Figure shows portion of an object model for a word processing
 - A document consists of many paragraphs, each of which consists of many sentences.



50

- Eg:2
- The class *Consortium* is the aggregation of its member banks.

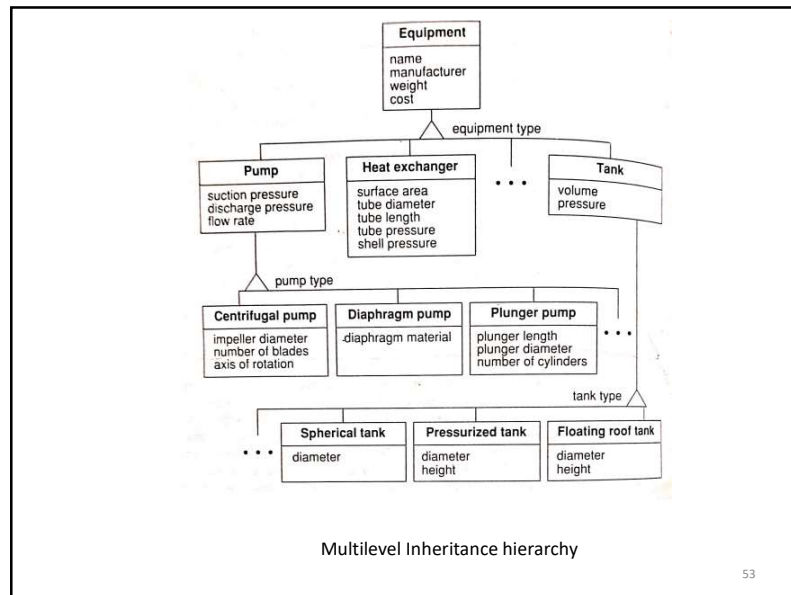


51

• Generalization and Inheritance:

- Generalization is the relationship between a class and one or more refined versions of it.
- The class being refined is called the ‘superclass’ and each refined version are called a ‘subclass’.
- Eg: Equipment is the superclass of Pump and tank.
- Attributes and operations that are common to a group of subclasses are attached to the superclass and shared by each subclass.
- Generalization is sometimes called a ‘is-a’ relationship.
- Each instance of a subclass is an instance of the superclass.
- The notation for generalization is a triangle connecting a superclass to subclass.

52

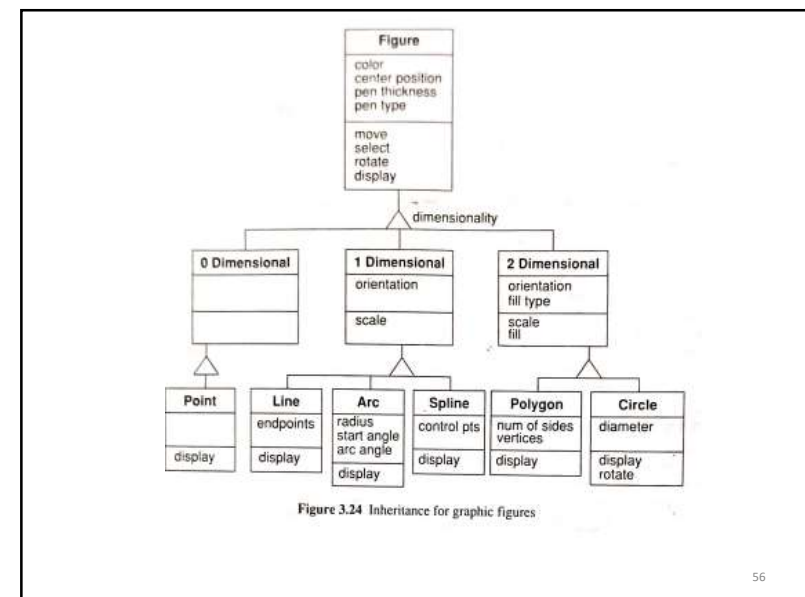


- Figure shows an equipment generalization.
- Each piece of equipment is a pump, heat exchanger, tank or another type of equipment
- There are several kinds of pumps: centrifugal, diaphragm, and plunger.
- There are several kinds of tanks: floating roof, pressurized and spherical.
- Pump type and tanktype both refine second level generalization classes down to a third level

54

- The words written next to triangles in the diagram, such as equipment type, pump type and tank type, are **discriminators**.
- A discriminator is an attribute of enumeration type that indicates which **property of an object is being abstracted by a particular generalization relationship**.
- Only one property should be discriminated at once.
- The discriminator is simply a name for the basis of generalization.

55



Use of Generalization

- Generalization has three purposes –
- 1. To **support polymorphism**: You can call an operation at the super class level, and the object oriented language compiler automatically resolves the call to the method that matches the calling object's class.
- 2. To **structure the description of objects**: i.e. to frame a taxonomy and organizing objects on the basis of their similarities and differences.
- 3. To **enable reuse of code**: Reuse is more productive than repeatedly writing code from scratch.

57

Grouping Constructs:

a) Module

- A module is a **logical construct** for **grouping classes, associations, and generalizations**.
- It **captures one perspective or view of a situation**.
- Eg: Electrical, plumbing, etc. are different views of a building.
- The **boundaries** of a module are somewhat **arbitrary** and **subject to judgment**.
- An **object model** consists of **one or more modules**.
- Modules enables you to **partition** an **object model** into **manageable pieces**.
- **Class** names and **association** names must be **unique** within a module.
- Module names are usually listed at the **top** of **each sheet**.
- The **same class** may be referenced **in many modules**.

58

Grouping Constructs:

b) Sheet

- A sheet is a **mechanism** for **breaking a large object model down into a series of pages**.
- Sheet is a **single printed page**.
- Each **module** consists of **one or more sheets**.
- Each **sheet** has a **title** and a **name or number**.
- Each **association/generalization** appears on a **single sheet**.
- **Classes** may appear on **multiple sheets**.

59

- Multiple copies of the same class form the bridge for connecting sheets in an object model.
- Sheet numbers or sheet names inside circle contiguous to a class box indicates other sheets that refer to a class.
- As a rule we never put more than one module per sheet.
- A sheet is just a **notational convenience**, not a logical construct

60

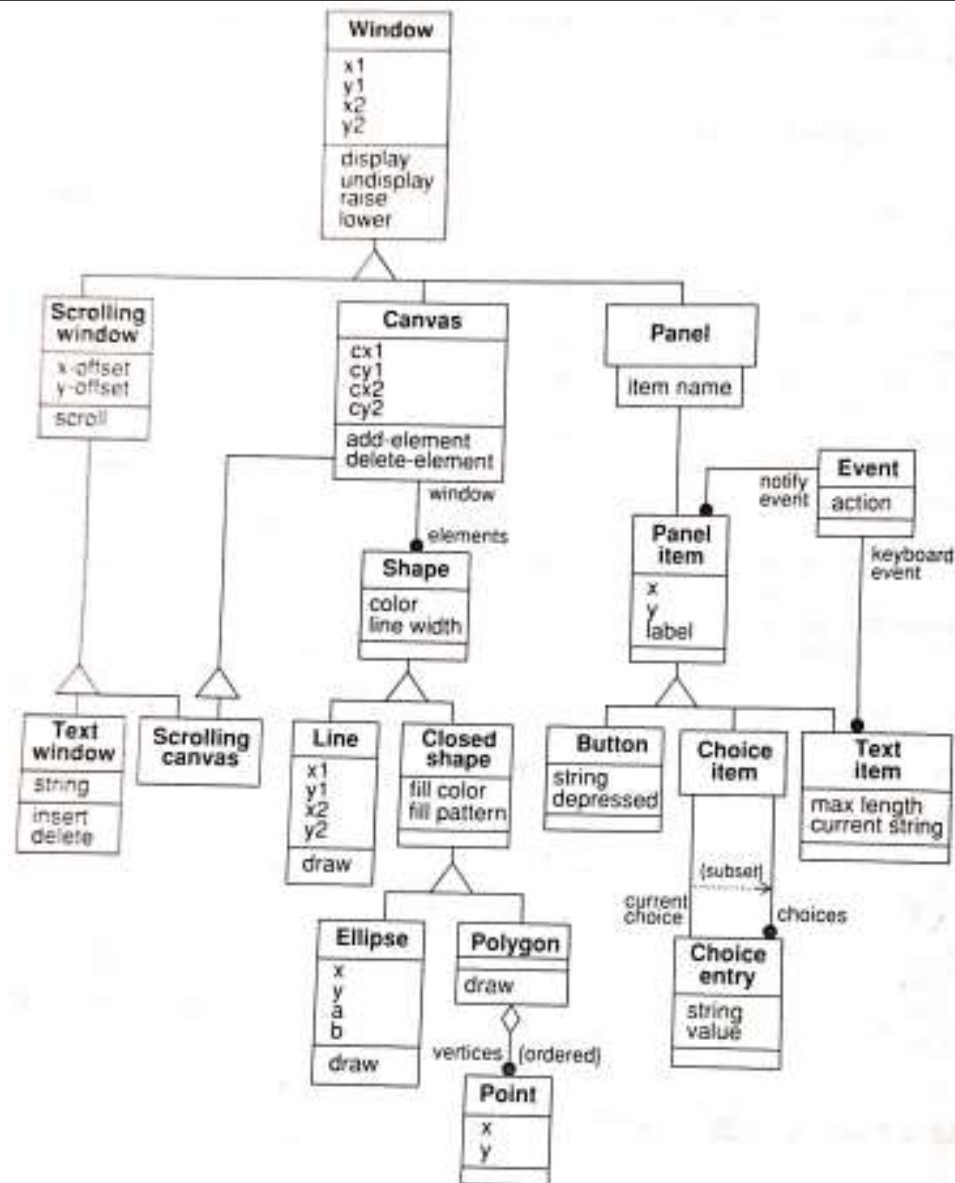


Figure 3.25 Object model of windowing system

- The above figure shows sample model of windowing system.
- Class window defines common parameters of all kinds of windows, including a rectangular boundary defined by the attributes x1, y1, x2, y2, and operations to display and undisplay a window and to raise it to top(fore ground) or lower it to bottom(background) of the entire set of windows.
- Panel, Canvas, and text window(type of scrolling window) are varieties of windows. A canvas is a region for drawing graphics. It inherits the window boundary from window and adds the dimensions of the underlying canvas region defined by the attributes cx1, cy1, cx2, cy2.
- A canvas contains a set of elements , shown by association to class shape. All shapes have color and line width. Shapes can be line, ellipses, or polygons, each with their own parameters.
- A polygon consist of an ordered list of vertices, shown an aggregation of many points. Ellipses and polygons are both closed shapes, which have a fill color and fill pattern. Lines are one dimensional and cannot be filled.

62

- Canvas windows have operations to add elements and to delete elements.
- Text window is a kind of scrolling window, which has two dimensional scrolling offset within its window, as specified by x-offset and y-offset, as well as an operation scroll to change the scroll value.
- A text window contains a string, and has operations to insert and delete characters. Scrolling canvas is a special kind of canvas that supports scrolling:it is both a canvas and a scrolling window. This is an example of multiple inheritance.

63